

Take-Home Assessment – Full-Stack Engineer

Overview

This take-home is designed to be completed in **approximately 3–4 hours**. It evaluates your system design thinking and your ability to build a small but functional full-stack feature in a realistic healthcare context.

A basic, working implementation is sufficient. We do **not** expect production-grade robustness, full test coverage, or pixel-perfect UI.

What we look for

- Clarity of system design and trade-offs
- API design and backend structure
- Frontend structure and UX basics
- Code readability and organisation
- Sensible use of libraries and framework features
- How you reason about security, privacy, and scalability (discussed in the interview)

1. System Design Phase

Design a system architecture for a **Healthcare Platform** that allows **authorised users** to manage patient health records and prescriptions, and track prescription statuses.

Please provide:

1. **Architecture diagram** (PDF or image)
 - Show the main components (frontend, backend/API, database, auth, any external services) and how they interact.
2. **Data model (high level)**
 - Define basic entities and relationships for:
 - Patient

- Prescription
- MedicalReport (patient vital reports)
- User/Role (e.g., Doctor, Pharmacist, Admin)

3. Design notes (3–10 bullet points)

- Briefly describe your approach to:
 - Authentication and authorisation (e.g., how users are identified and what they can access).
 - Handling sensitive health data (e.g., transport security, storage, auditability at a conceptual level).
 - Scalability and performance (e.g., stateless APIs, database indexing, possible caching).

This does not need to be long; a clear diagram and concise bullets are enough.

2. Backend Requirements

Use **Python for the backend** (e.g., Django, FastAPI, Flask) and **React for the frontend**. Please structure your projects as you would for a production-ready application using these technologies.

Implement the following:

- Create a Patient Record API to register a new patient in the system.
- Create an Update Patient Record API to modify existing patient information.
- Create a Fetch All Patient Records API with pagination, search, and filters (name, age, gender).
- Create a Prescription API to add a prescription for a specific patient.
- Create an Update Prescription Status API to change the prescription status (Pending, Approved, Dispensed).
- Create an API to fetch all prescriptions with status = Pending.
- Implement middleware to validate session tokens for protected APIs.

3. Frontend Requirements

Create a **separate frontend project** that talks to your backend APIs.

Implement the following:

- Build a Patient Registration Component with form validation.
- Build a Patient List Component that displays patients with pagination and filters.
- Build a Patient Details Component to show the patient profile and prescription history.
- Implement Search and Filter UI for patients.

4. Submission Instructions

You can submit your work in one of the following ways:

- A link to a **GitHub repository** containing both frontend and backend projects, or
- A **ZIP file** containing both projects.
- PDF